Crashing and Recovering: Fail-Fast Resilience Experiment

1. Introduction

In this experiment, I simulated a downstream dependency failure inside my ECS-hosted product search API and evaluated how the system behaved under load. The goal was to demonstrate how a poorly handled dependency can degrade system performance, and how implementing a fail-fast timeout improves resilience and throughput.

The system is deployed using:

- Amazon ECS (Fargate)
- Application Load Balancer (ALB)
- Auto Scaling enabled
- CloudWatch for monitoring
- Locust for load testing

The `/products/search` endpoint was modified to simulate a slow downstream dependency.

2. Problematic Deployment (Broken Version)

2.1 Simulated Failure

I introduced a fake downstream service inside the `/products/search` handler:

```
func fakeExternalService() error {
    if rand.Intn(100) < 30 {
        time.Sleep(2 * time.Second)
        return errors.New("downstream timeout (simulated)")
    }

    time.Sleep(10 * time.Millisecond)
    return nil
}
```

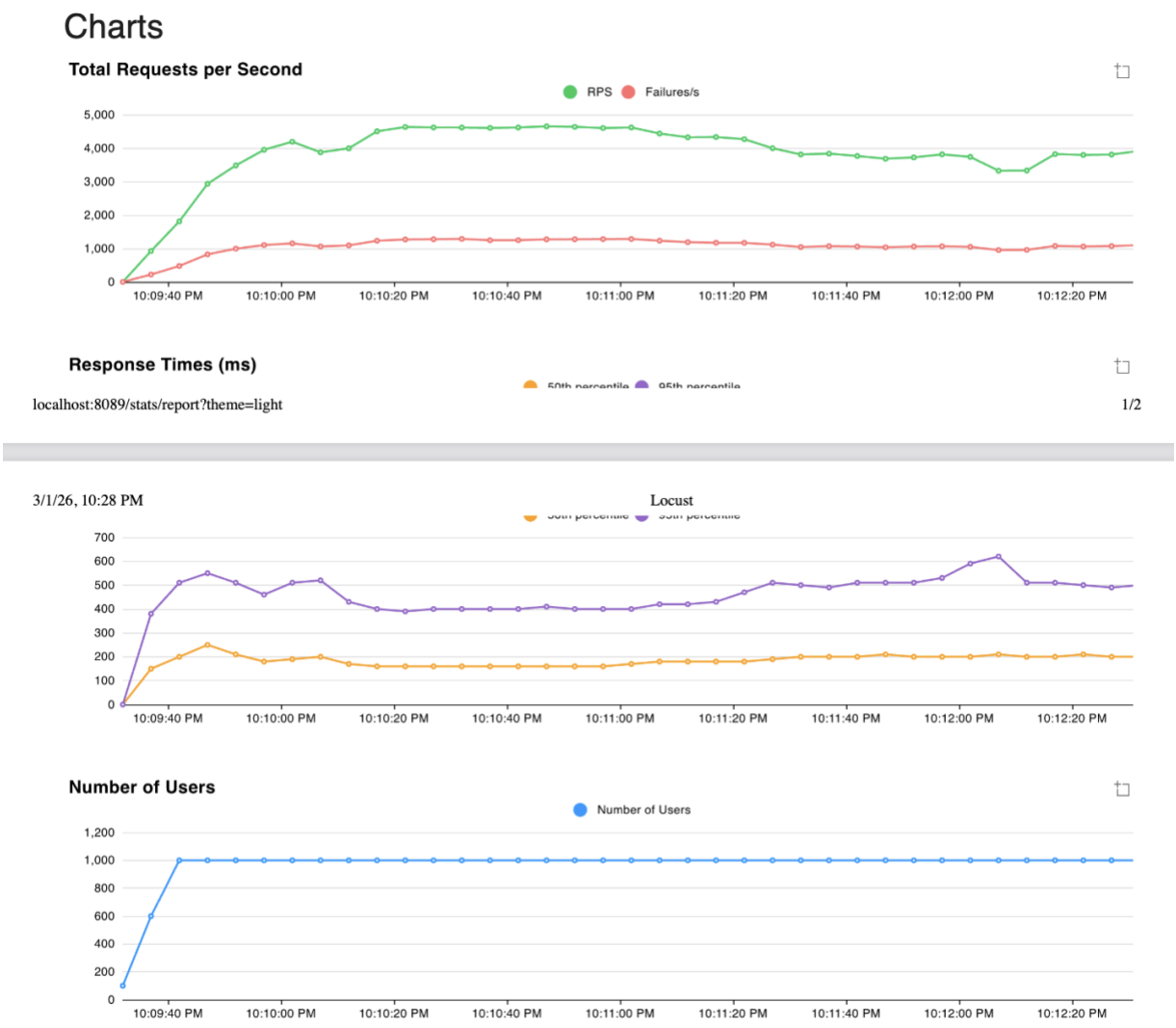
In this configuration:

- 30% of requests sleep for 2 seconds

- The API waits the full duration
- This causes request blocking and queue buildup under load

2.2 Load Test Configuration

- 1000 concurrent users
- 100 requests per second
- 3–5 minute test duration
- Traffic routed through ALB



Locust Test Report

During: 3/1/2026, 7:00:17 PM - 3/1/2026, 7:03:17 PM (3 minutes)
Target Host: http://product-alb-588149756.us-west-2.elb.amazonaws.com
Script: locustfile.py

Request Statistics

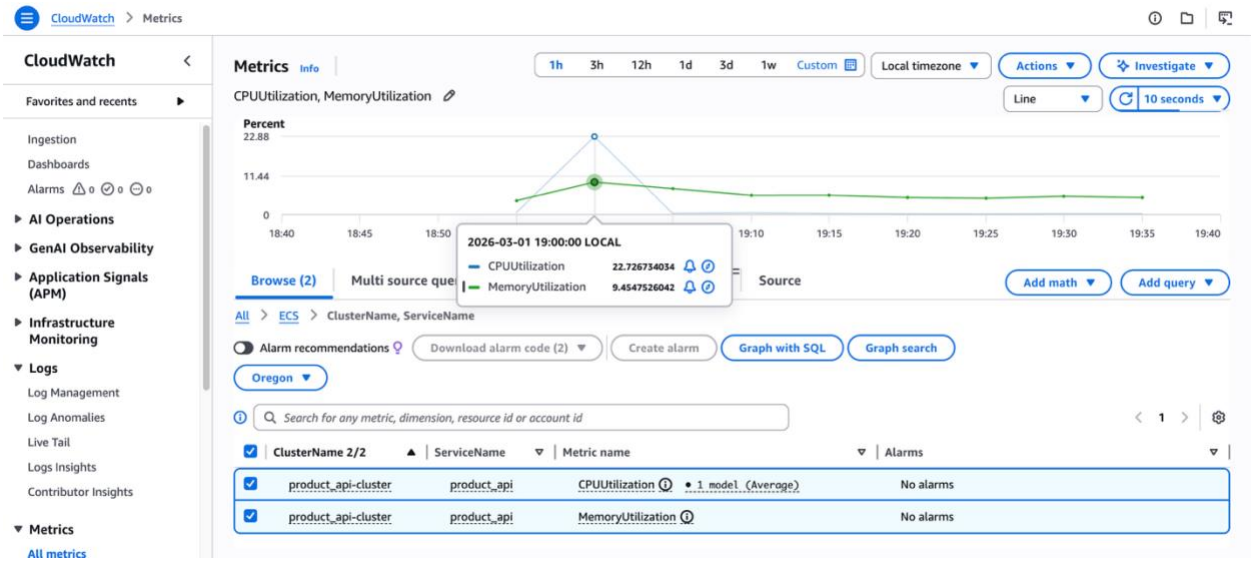
Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	GET /health	24232	0	100.26	77	725	15	134.55	0
GET	GET /products/search	242399	71889	704.32	87	2728	701.49	1345.98	399.18
	Aggregated	266631	71889	649.42	77	2728	639.1	1480.53	399.18

Response Time Statistics

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
GET	GET /health	95	97	98	100	100	120	220	730
GET	GET /products/search	110	110	330	2100	2100	2100	2200	2700
	Aggregated	110	110	170	2100	2100	2100	2200	2700

Failures Statistics

# Failures	Method	Name	Message
71889	GET	GET /products/search	CatchResponseError('Expected 200, got 503')



2.3 Observed Behavior (Broken)

Under load, the system exhibited:

- Aggregated RPS: **~1480 RPS**
- High response times due to 2-second blocking
- Increased request queuing
- Controlled 503 errors
- Moderate CPU utilization

Even though CPU was not fully saturated, throughput was significantly limited because requests were blocked waiting on the slow dependency.

This demonstrates a classic cascading latency problem in distributed systems.

3. Implemented Fix: Fail-Fast Timeout

To improve resilience, I wrapped the dependency call with a timeout:

```go
func fakeExternalServiceWithTimeout(timeout time.Duration) error {
    done := make(chan error, 1)

    go func() {
        done <- fakeExternalService()
    }()

    select {
    case err := <-done:
        return err
    case <-time.After(timeout):
        return errors.New("dependency timed out fast (fail-fast)")
    }
}
```

```go
// var depErr = fakeExternalService()
var depErr = fakeExternalServiceWithTimeout(200 * time.Millisecond)
if depErr != nil {
// Controlled failure: we return fast instead of hanging
        c.JSON(http.StatusServiceUnavailable, gin.H{
```

```
        "error":  "dependency failed",
        "detail": depErr.Error(),
    })
    return
}
```

Instead of waiting 2 seconds, the system now:

- Stops waiting after 200ms
- Returns 503 immediately
- Frees worker capacity
- Prevents request pile-up

This implements the **Fail-Fast resilience pattern**.

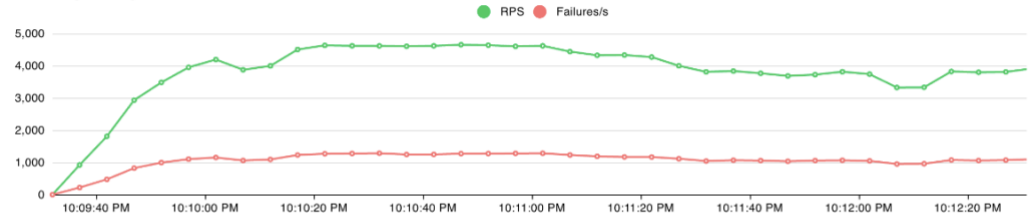
4. Improved Deployment (Fixed Version)

4.1 Load Test Configuration (Same as Before)

- 1000 users
- 100 req/s
- Same duration
- Same infrastructure
- Only code behavior changed

Charts

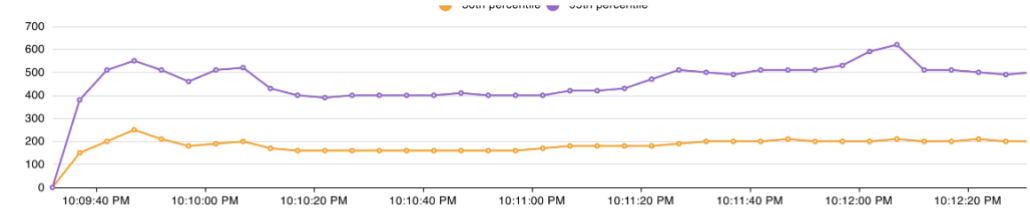
Total Requests per Second



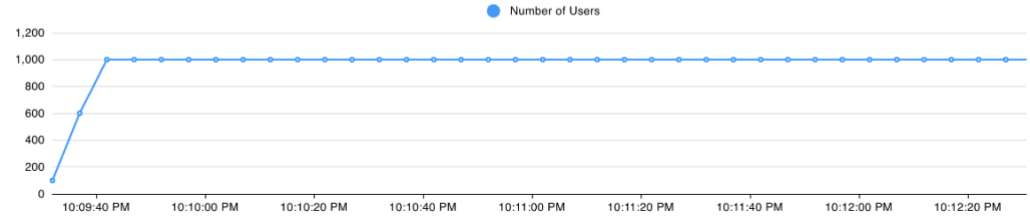
Response Times (ms)

1/26, 10:28 PM

Locust



Number of Users



Locust Test Report

Download the Report

During: 3/1/2026, 10:09:31 PM - 3/1/2026, 10:12:35 PM (3 minutes and 4 seconds)
Target Host: http://product-alb-7095173.us-west-2.elb.amazonaws.com
Script: locustfile.py

Request Statistics

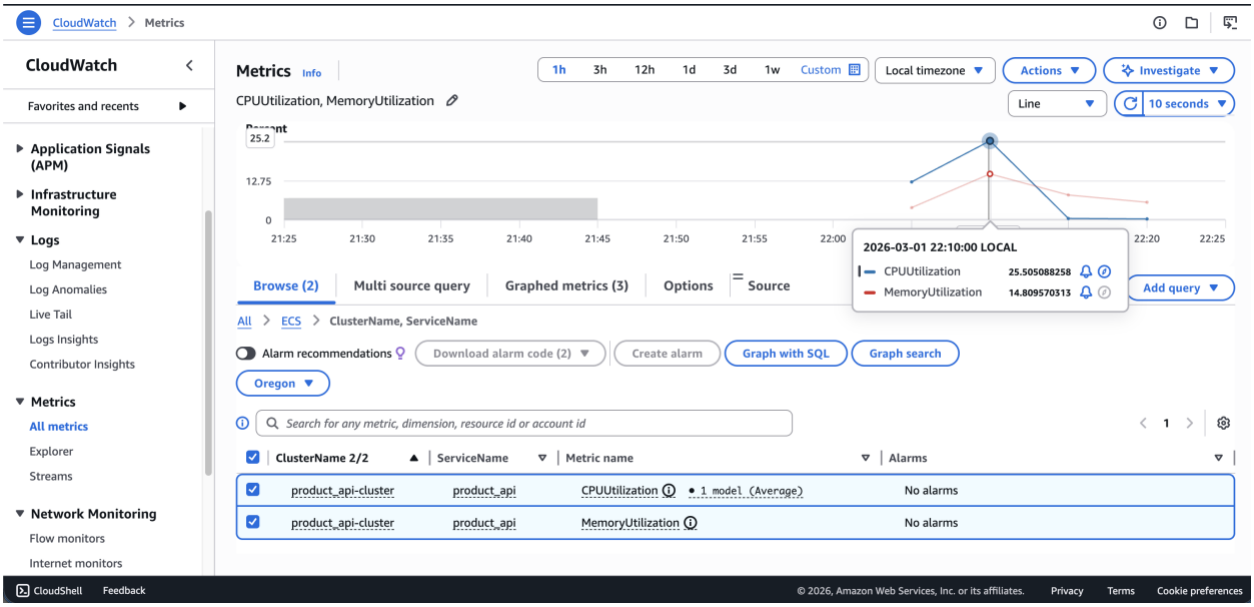
Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	GET /health	65925	0	155.63	79	1337	15	358.24	0
GET	GET /products/search	665728	202778	242.73	89	1775	620.89	3617.61	1101.91
	Aggregated	731653	202778	234.88	79	1775	566.29	3975.85	1101.91

Response Time Statistics

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
GET	GET /health	120	150	170	190	250	350	490	1300
GET	GET /products/search	200	270	310	350	410	490	640	1800
	Aggregated	190	230	300	340	400	480	630	1800

Failures Statistics

# Failures	Method	Name	Message
202778	GET	GET /products/search	CatchResponseError('Expected 200, got 503')



4.2 Observed Behavior (Fixed)

Under identical load conditions:

- Aggregated RPS increased from **~1480 -> ~3975**
- Response times significantly improved
- Throughput stabilized
- CPU slightly increased (~25%) due to higher throughput
- Failures remained controlled and predictable (503)

The increase in CPU utilization reflects improved throughput and system activity, not instability.

5. Before vs After Comparison

Metric	**Broken**	**Fixed**
RPS	~1480	~3975
Dependency Wait	2 seconds	200ms
Latency	High / Blocking	Bounded
Throughput	Limited	Scaled
Failure Type	Slow degradation	Fast controlled failure

The fixed system demonstrates graceful degradation instead of cascading slowdown.

6. Conclusion

This experiment demonstrates how improper handling of downstream slowness can significantly reduce system throughput without necessarily saturating CPU resources.

By implementing a fail-fast timeout:

- Request blocking was eliminated
- Throughput nearly tripled
- Latency stabilized
- The system became more resilient under load

This validates the importance of resilience patterns such as **Fail Fast** in distributed systems.

The experiment successfully demonstrated:

- A problematic deployment
- A resilience-based fix
- Measurable improvement through controlled testing
- Use of CloudWatch metrics to evaluate system behavior